

实验六 分类算法（3）

（一）目的和要求

- （1）掌握分类算法中的支持向量机的概念的使用场景
- （2）掌握使用支持向量机的方法和步骤
- （3）掌握模型参数优化的方法

（二）实验知识准备

支持向量机(Support Vector Machine, SVM): 是一种监督学习模型, 广泛应用于分类和回归任务。其主要作用是通过构建一个超平面, 将不同类别的数据点分隔开, 以最大化类别之间的间隔。这种方法特别适合于高维数据, 并能够有效处理非线性问题(通过使用核函数)。在计算生物学中, SVM 常用于基因表达数据分析、蛋白质分类、非编码 RNA 识别等任务, 因其出色的分类性能和较强的泛化能力。

（1）最大间隔分割: SVM 寻找一个超平面, 使得不同类别的数据点之间的间隔最大化。这种最大间隔的策略有助于提高分类器的泛化能力。

（2）支持向量: SVM 仅依赖于靠近决策边界的数据点(称为支持向量)来构建模型, 而不是使用所有训练数据, 从而降低计算复杂性并增强模型鲁棒性。

（3）核函数: SVM 通过核函数将数据映射到高维空间, 以便处理非线性可分问题。常用的核函数如下:

a. 线性核函数, 线性核函数是 SVM 中最简单的核函数之一, 它将输入数据直接映射到高维空间, 计算两个样本之间的内积来衡量它们之间的相似度。线性核函数在处理线性可分问题时表现良好, 但在处理非线性可分问题时效果较差。

b. 多项式核函数, 多项式核函数通过将输入数据映射到高维空间, 并使用多项式函数来计算样本之间的相似度。多项式核函数能够处理一定程度的非线性关系, 但在高维空间中计算复杂度较高, 需要注意选择合适的多项式阶数。

c. 高斯核函数, 高斯核函数是 SVM 中最常用的核函数之一, 也称为径向基函数(Radial Basis Function, RBF)核函数。它通过将输入数据映射到无穷维的高维空间, 并使用高斯函数来计算样本之间的相似度。高斯核函数能够处理复杂的非线性关系, 并且具有较好的鲁棒性和泛化能力。

d. sigmoid 核函数，sigmoid 核函数是 SVM 中的一种非常特殊的核函数，它将输入数据映射到高维空间，并使用 sigmoid 函数来计算样本之间的相似度。sigmoid 核函数主要用于二分类问题，并且在神经网络中有广泛的应用。

在实际应用中，选择合适的核函数对 SVM 的性能和泛化能力至关重要。对于线性可分问题，线性核函数通常是最好的选择；对于非线性可分问题，高斯核函数是一个常用的选择。此外，多项式核函数和 sigmoid 核函数在特定的问题中也有一定的应用价值。

(4) 正则化：SVM 通过引入正则化参数（C），在最大化间隔和降低分类错误之间进行权衡，帮助防止过拟合。

(5) 适用性广泛：SVM 可以用于二分类和多分类问题，且在处理高维数据时表现优异，广泛应用于图像识别、生物信息学和文本分类等领域。

(6) 除了核函数的选择，SVM 还需要设置其他参数，如惩罚因子 C 和松弛变量的上界。这些参数的合理选择对于 SVM 的性能和泛化能力同样至关重要。在实际应用中，可以使用交叉验证等方法来选择最优的参数组合。

（三）实验内容及步骤

(1) 随机生成一组线性可分数据，数据格式为 (x1,x2)，同时数据划分为 2 类，分别为 0 和 1。比如样本 (x1,x2) 属于 0 类。使用以上数据训练支持向量机模型，得到最优的超平面，输出模型参数 W 和 b。并对新出现的样本，预测样本属于哪个类别。

1. 步骤一，导入本项目所需要的模块和包，如图 1 所示。

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn import svm
4 from sklearn.datasets import make_blobs
```

图 1 导入所需模块

2. 步骤二，随机生成数据，随机生成 40 个样本数据，数据分为两类。数据格式为 (x1,x2,y)，x1、x2 为数据的特征，y 是数据所属类别。参考代码如图 2 所示。

```

1 X, y = make_blobs(n_samples=40, centers=2, random_state=6)
2 # 可视化数据
3 plt.figure(figsize=(8, 6))
4 plt.scatter(X[:, 0], X[:, 1], c=y, cmap='viridis', marker='o', edgecolor='k', s=100)
5 plt.title('Visualization of Generated Data')
6 plt.xlabel('X1')
7 plt.ylabel('X2')
8 plt.grid(True)
9 plt.show()

```

图 2 生成数据

其中，使用 `make_blobs(n_samples=40, centers=2, random_state=6)` 方法生成 40 个线性可分的样本数据，分为 2 个中心。

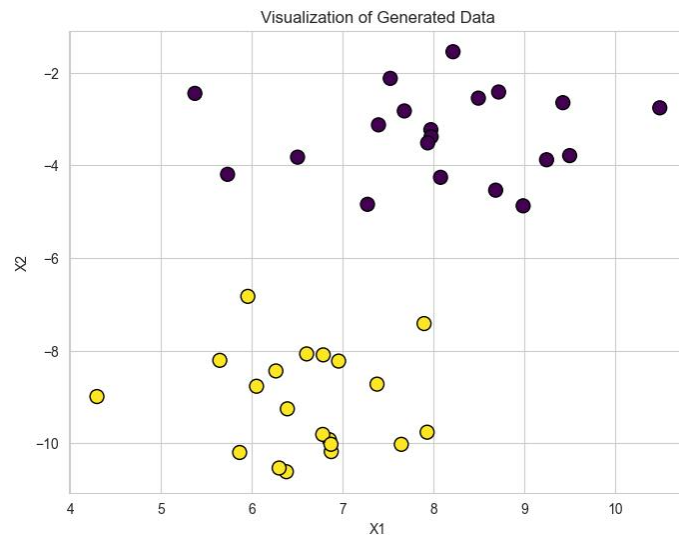


图 3 数据分布

3. 步骤三，训练 SVM 模型，建立好一个未训练的 SVM 模型，将训练样本传给模型进行训练，主要代码如图 4 所示。

```

1 clf = svm.SVC(kernel='linear', C=1000)           #SVC为SVM的一种类型, 是用来做分类的
2 clf.fit(X, y)                                    #喂给模型数据, 进行训练
3 print("模型参数W:", clf.coef_)                  #模型参数W
4 print("模型参数b:", clf.intercept_)             #模型参数b
5 print("支持向量为:", clf.support_vectors_)      #输出支持向量

```

图 4 训练 SVM

其中，SVC 为 Support Vector Classification，支持向量机用于分类，除此之外，还有 Support Vector Regression(SVR)支持向量机用于回归分析。

4. 步骤四，使用新数据进行预测，示例代码如图 5 所示。

```

1 xpredict = [10, -10]
2 xpredict = np.array(xpredict).reshape(1, -1)
3 print(xpredict, "预测为:", clf.predict(xpredict), "类别")
4 xpredict = [10, -2]
5 xpredict = np.array(xpredict).reshape(1, -1)
6 print(xpredict, "预测为:", clf.predict(xpredict), "类别")

```

图 5 定义并训练分类器

5. 步骤五，使用 `matplotlib` 库进行绘图，通过将间隔、划分超平面和支持向量描绘出来，示例代码如图 6。

```

1 # 可视化数据点
2 plt.figure(figsize=(12, 8))
3 scatter = plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired, alpha=0.7, edgecolors='k')
4
5 # 在每个数据点上添加坐标和分类标签
6 for i in range(len(X)):
7     plt.text(X[i, 0] + 0.05, X[i, 1] + 0.05, f'({X[i, 0]:.2f}, {X[i, 1]:.2f})\nLabel: {y[i]}',
8             fontsize=9, ha='left', va='bottom', color='black', bbox=dict(facecolor='white', alpha=0.5,
9                 edgecolor='none', boxstyle='round, pad=0.3'))
10 # 绘制决策边界
11 ax = plt.gca()
12 xlim = ax.get_xlim()
13 ylim = ax.get_ylim()
14 xx = np.linspace(xlim[0], xlim[1], 30)
15 yy = np.linspace(ylim[0], ylim[1], 30)
16 YY, XX = np.meshgrid(yy, xx)
17 xy = np.vstack([XX.ravel(), YY.ravel()]).T
18 Z = clf.decision_function(xy).reshape(XX.shape)
19 # 绘制分割界
20 contour = ax.contour(XX, YY, Z, colors='k', levels=[-1, 0, 1], alpha=0.5, linestyles=['--', '-', '--'])
21 # 在决策边界上添加坐标
22 for i, level in enumerate(contour.levels):
23     # 找到等高线的坐标
24     for collection in contour.collections:
25         for path in collection.get_paths():
26             vertices = path.vertices
27             # 取中点添加坐标标签
28             mid_point = vertices[len(vertices) // 2]
29             plt.text(mid_point[0], mid_point[1], f'({mid_point[0]:.2f}, {mid_point[1]:.2f})',
30                     fontsize=10, ha='center', va='center')
31 # 绘制支持向量
32 ax.scatter(clf.support_vectors_[:, 0], clf.support_vectors_[:, 1], linewidth=1, facecolors='red', label='Support
    Vectors')

```

图 6 分类结果可视化

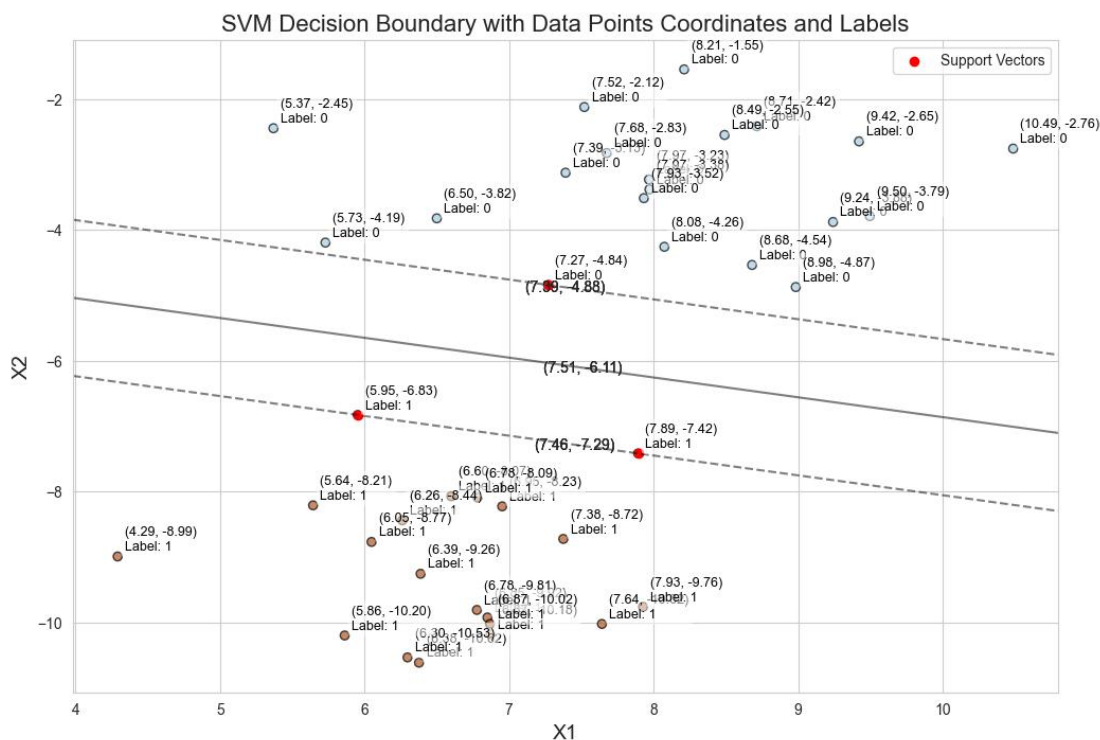


图 7 可视化结果

图中每个边界都对应一个坐标点（如“(7.39, -4.88)”和“(5.64, -6.83)”等），这些坐标点是支持向量的位置。支持向量是那些离决策边界最近的点，对于确定决策边界的位置至关重要。每个决策边界旁边都标有“Label: 0”或“Label: 1”，这表示了边界两侧数据点的类别

(2) 随机生成一组线性不可分数据，数据格式为 (x_1, x_2, x_3) ，同时数据划分为 2 类，分别为 0 和 1。比如样本 (x_1, x_2, x_3) 属于 0 类。

6. 步骤六，生成线性不可分数据，示例代码如图 8 所示。

```
1 from sklearn.datasets import make_moons
2 # 生成线性不可分的数据
3 X, y = make_moons(n_samples=100, noise=0.1, random_state=6)
4 # 添加一个随机的第三维度
5 Z = np.random.rand(X.shape[0]) # 随机生成三维数据
6 X_3d = np.column_stack((X, Z)) # 合并成三维数据
7 # 可视化三维数据点
8 fig = plt.figure(figsize=(14, 8))
9 ax = fig.add_subplot(111, projection='3d')
10 scatter = ax.scatter(X_3d[:, 0], X_3d[:, 1], X_3d[:, 2], c=y, cmap=plt.cm.Paired, alpha=0.7)
11 ax.set_xlabel('X1', fontsize=10)
12 ax.set_ylabel('X2', fontsize=10)
13 ax.set_zlabel('X3', fontsize=10)
14 # 调整视角
15 ax.view_init(elev=40, azim=40)
16 plt.show()
```

图 8 生成数据

使用 `make_moons` 生成线性不可分的数据，`noise` 参数增加了数据的复杂性。

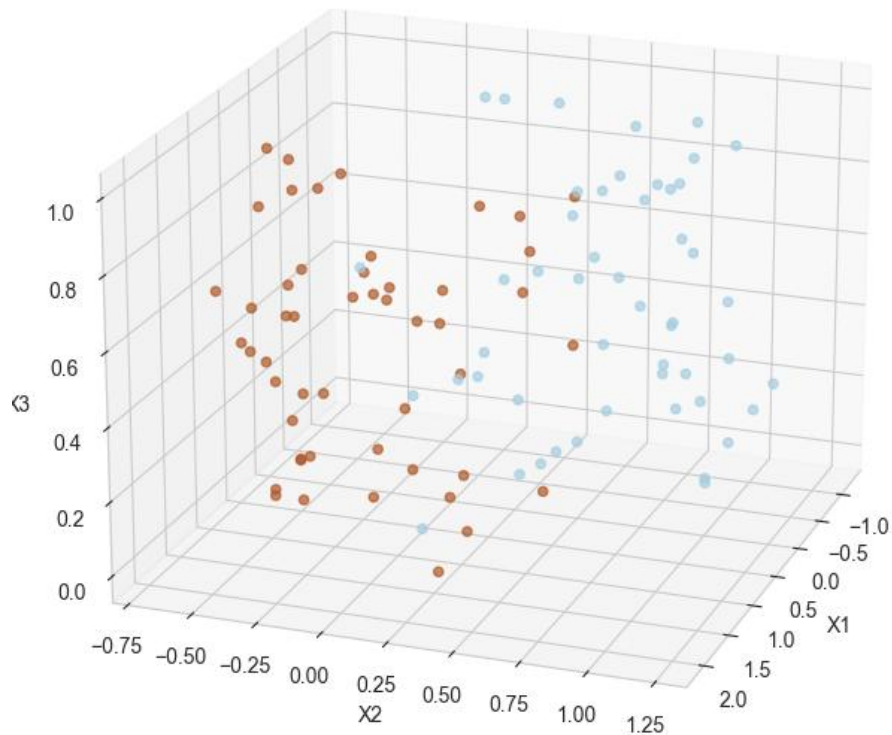


图 8 可视化数据分布

7. 步骤七，对选择后的模型，输出分类结果的评价指标，示例代码如图 9 所示。

```

1 # 训练支持向量机模型，使用RBF核
2 clf = svm.SVC(kernel='rbf', C=1)
3 clf.fit(X_3d, y)
4 # 可视化三维数据点
5 fig = plt.figure(figsize=(12, 8))
6 ax = fig.add_subplot(111, projection='3d')
7 scatter = ax.scatter(X_3d[:, 0], X_3d[:, 1], X_3d[:, 2], c=y, cmap=plt.cm.Paired, alpha=0.7)
8 #绘制决策边界（超平面）
9 xx, yy = np.meshgrid(np.linspace(X_3d[:, 0].min(), X_3d[:, 0].max(), 20),
10                      np.linspace(X_3d[:, 1].min(), X_3d[:, 1].max(), 20))
11 zz = np.zeros(xx.shape)
12 #计算决策函数值
13 for i in range(xx.shape[0]):
14     for j in range(xx.shape[1]):
15         z = np.array([[xx[i, j], yy[i, j]]]) #这里可以设置Z为0以便于绘制
16         zz[i, j] = clf.decision_function(z)
17 #绘制决策超平面
18 ax.contour3D(xx, yy, zz, levels=[0], colors='k', linewidths=2)
19 #设置图形标题和标签
20 ax.set_xlabel('X1', fontsize=10)
21 ax.set_ylabel('X2', fontsize=10)
22 ax.set_zlabel('X3', fontsize=10)
23 plt.show()

```

图 9 主要代码

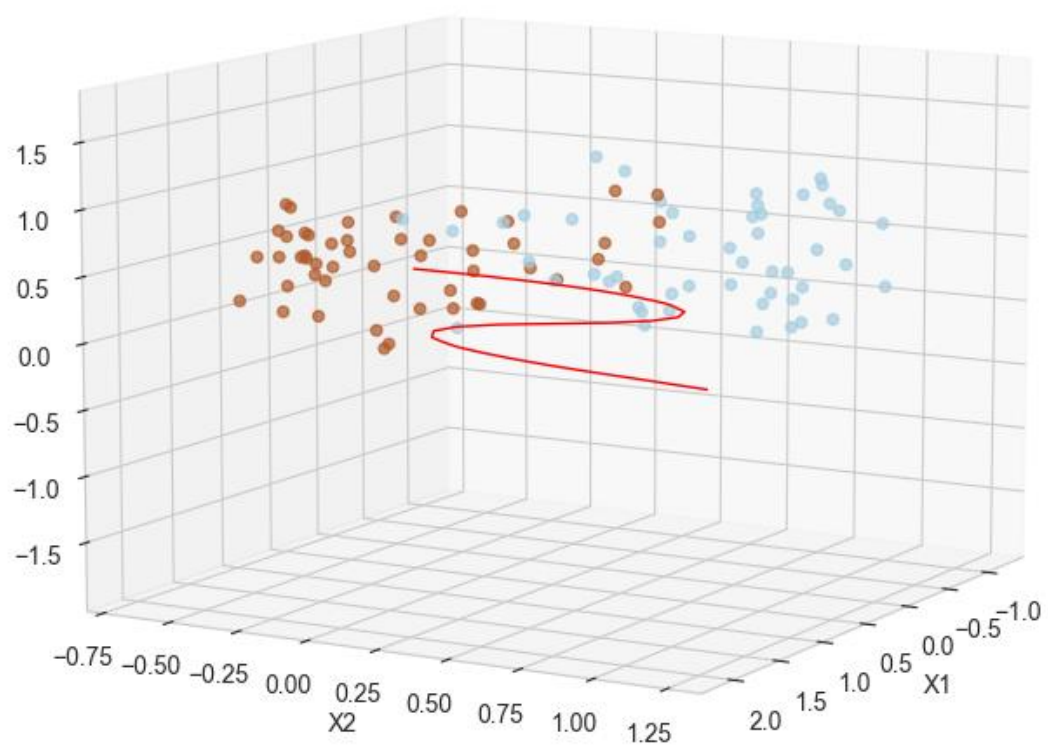


图 9 可视化线性不可分

(1) 实训项目

使用支持向量机或其他机器学习方法，自选研究背景和数据，完成一个小型的数据分类问题：包括但不限于以下过程：

1. 项目背景，以解决实际问题为本实训项目的背景，可参考实验教程或其他资料。
2. 数据集选择，公开数据集或者实验中用到的数据集。
3. 实验设计应包括一下几个具体步骤

a. 数据预处理：加载数据集，划分为训练集和测试集。对数据进行归一化或标准化处理。通过可视化的方式查看数据分布情况。

b. 特征选择：由于数据集的特征维问题，考虑特征选择的方法选择出对预测结果对有效的特征，以减少计算复杂度和过拟合风险。使用特图进一步分析特征。

c. 模型选择：选择 SVM 或其他分类算法，并对模型进行参数寻优等优化过程。

d. 模型训练：使用优化后的参数重新训练模型，已达到最佳性能。

e. 性能评估：使用测试集数据评估模型的性能，计算准确率、召回率、F1 分数等评价指标。通过可视化方法（如混淆矩阵）进一步分析模型的分类效果。

f. 结果分析与讨论：分析模型在机器学习任务中的表现，讨论特征选择选择、参数优化等因素对模型性能的影响。

g. 比较 SVM 与其他机器学习算法（如神经网络、决策树等）在该任务上的性能差异，探讨各自的优缺点和适用场景。

h. 结论：对本次实训项目进行总结，并指出存在的问题。

4. 参考资料：

1. [Kaggle: Your Machine Learning and Data Science Community](#)
2. [数据科学科研和教学一体化平台 \(idatascience.cn\)](#)
3. [Examples — scikit-learn 1.5.2 documentation](#)
4. [GitHub: Let's build from here • GitHub](#)